

Home task: data engineering

Phase 0: Preparation and Financial Safety

1. AWS Free Tier Account Creation

An AWS Free Tier account was successfully created using an email address and a virtual card with a minimal balance for verification purposes.

To improve account security and avoid unexpected charges:

- Multi-Factor Authentication (MFA) was enabled.
- AWS Budget alerts were configured to monitor spending.
- Free Tier usage limits were reviewed prior to using any AWS services.

2. Billing Alarm Configuration

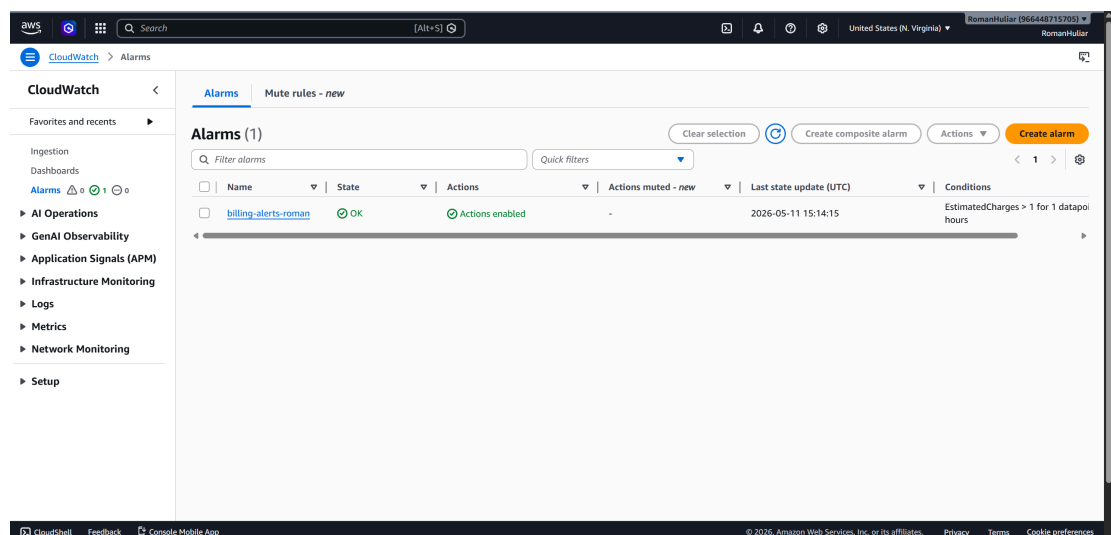
A CloudWatch Billing Alarm named `billing-alerts-roman` was created to monitor AWS charges.

The alarm was configured with the following condition:

- `EstimatedCharges > 1 USD`

Amazon SNS email notifications were configured to deliver billing alerts to the registered email address.

The SNS subscription was confirmed successfully via the email confirmation link.



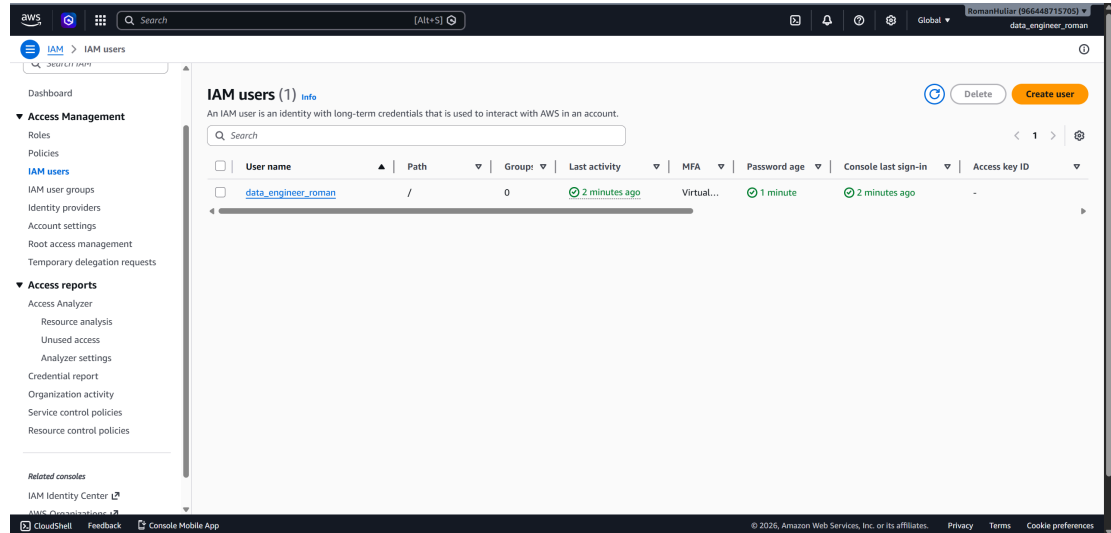
Phase 1: Identity and Access Management (IAM)

A new IAM user was created to work with AWS services instead of using the root account.

The user was named `data_engineer_roman`.

The following security and access settings were applied:

- AWS Management Console access was enabled.
- The `AdministratorAccess` policy was attached to the user.
- Multi-Factor Authentication (MFA) was enabled (Virtual MFA device).
- The initial sign-in was completed successfully using the IAM user credentials.

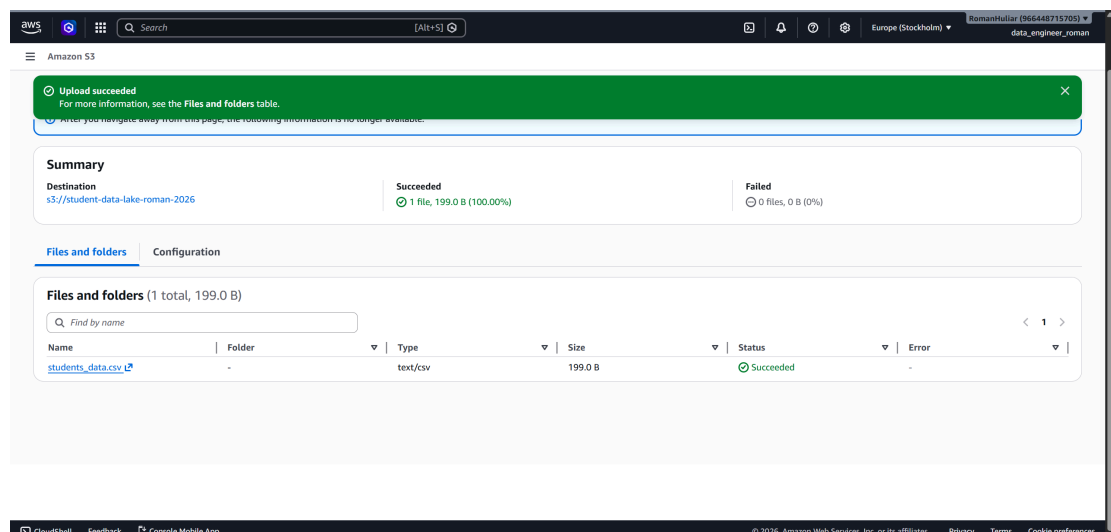


Phase 2: Building a Data Lake with S3, Glue and Athena

2.1 S3: Data Storage

An S3 bucket named `student-data-lake-roman-2026` was created in the **Europe (Stockholm)** region to store source data for the Data Lake pipeline.

A CSV file named `students_data.csv` was uploaded to the bucket successfully (199.0 B, 100% success rate).



2.2 IAM Role: Access for AWS Glue

An IAM role was created for AWS Glue to grant access to S3 and the Glue Data Catalog.

This role allows AWS Glue to read the uploaded CSV file from the S3 bucket and register metadata in the Glue Data Catalog.

The following policies were attached to the role:

- AmazonS3FullAccess
- AWSGlueServiceRole

This role was subsequently used by the Glue Crawler to scan the S3 data and automatically infer a table schema.

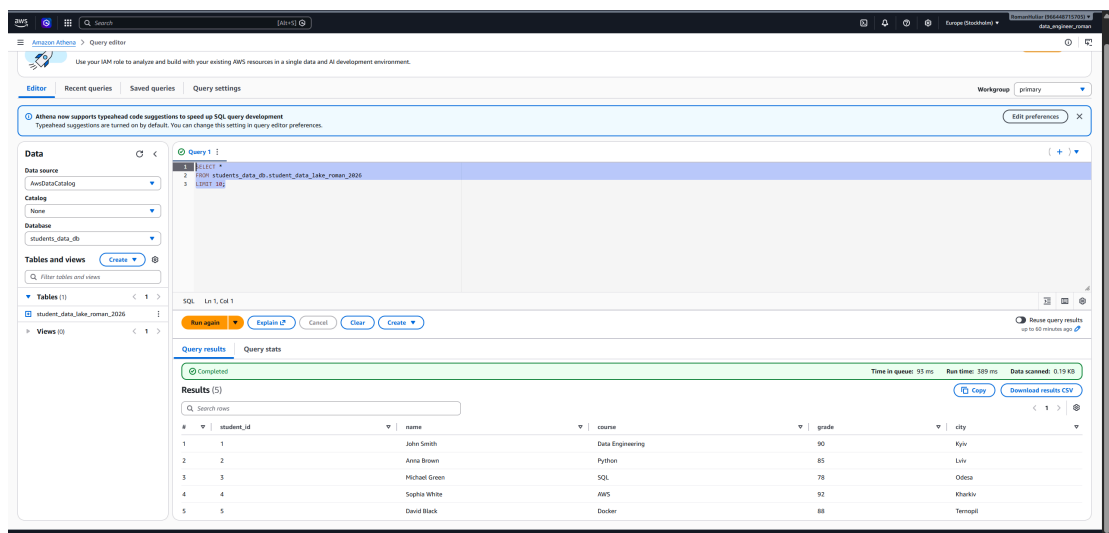
2.3 AWS Glue Crawler and Data Catalog

An AWS Glue Crawler was created and configured to scan the CSV file stored in the S3 bucket `student-data-lake-roman-2026`.

The crawler used the target database `students_data_db`. After running, AWS Glue automatically detected the schema and created a table in the Glue Data Catalog.

The resulting table was named `student_data_lake_roman_2026`. The schema includes the following columns:

- `student_id` — `bigint`
- `name` — `string`
- `course` — `string`
- `grade` — `bigint`
- `city` — `string`



The screenshot shows the Amazon Athena Query Editor interface. On the left, the 'Data' pane shows the data source as 'AwsDataCatalog', the catalog as 'None', and the database as 'students_data_db'. The 'Tables and views' pane shows a table named 'student_data_lake_roman_2026'. The main editor area contains a SQL query: `SELECT * FROM students_data_db.student_data_lake_roman_2026`. Below the query, the 'Query results' section shows the query is 'Completed'. The 'Results (5)' section displays a table with 5 rows and 5 columns: `student_id`, `name`, `course`, `grade`, and `city`. The data is as follows:

#	student_id	name	course	grade	city
1	1	John Smith	Data Engineering	90	Kyiv
2	2	Anna Brown	Python	85	Lviv
3	3	Michael Green	SQL	78	Odessa
4	4	Sophia White	AWS	92	Kharkiv
5	5	David Black	Docker	88	Terнопil

2.4 Amazon Athena: Querying Data

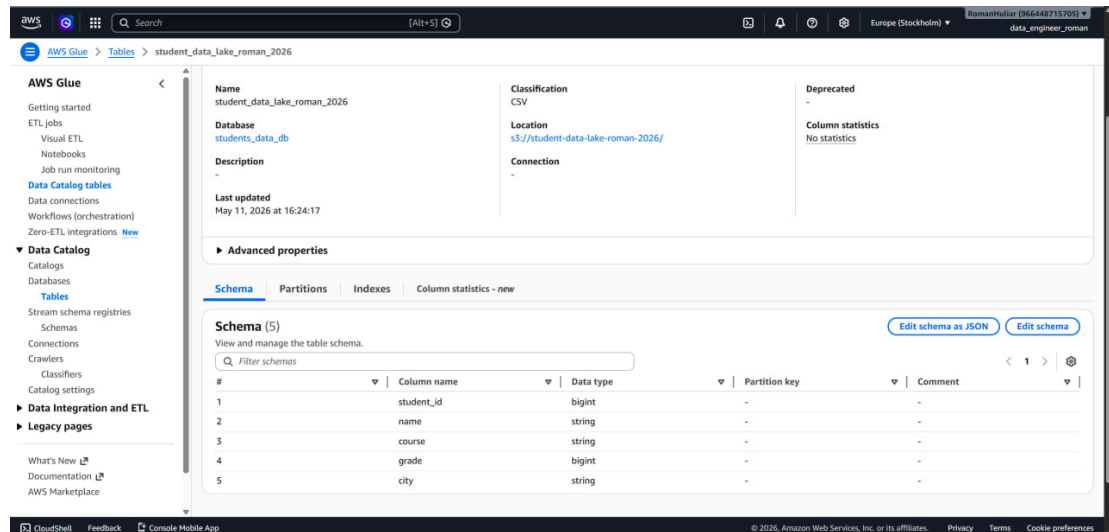
Amazon Athena was used to query the table created by the AWS Glue Crawler.

The database `students_data_db` and the table `student_data_lake_roman_2026` were selected in the Athena Query Editor.

The following SQL query was executed:

```
SELECT *
FROM students_data_db.student_data_lake_roman_2026
LIMIT 10;
```

The query completed successfully (Run time: 389 ms, Data scanned: 0.19 KB) and returned 5 rows with student records including fields: `student_id`, `name`, `course`, `grade`, and `city`.



Phase 3: LocalStack and AWS CLI (Optional)

As an optional part of the task, LocalStack was used to simulate AWS services locally through Docker and AWS CLI.

LocalStack was started in a Docker container using the image

`localstack/localstack:4.4.0`.

After the service became ready, AWS CLI commands were executed against the LocalStack endpoint:

```
aws --endpoint-url=http://localhost:4566 s3 mb s3://test-bucket
aws --endpoint-url=http://localhost:4566 s3 cp data.csv s3://test-bucket
aws --endpoint-url=http://localhost:4566 s3 ls s3://test-bucket
```

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved!

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\User> wsl
user@LAPTOP-H03GEGCQ:/mnt/c/Users/User$ export AWS_ACCESS_KEY_ID=test
user@LAPTOP-H03GEGCQ:/mnt/c/Users/User$ export AWS_SECRET_ACCESS_KEY=test
user@LAPTOP-H03GEGCQ:/mnt/c/Users/User$ export AWS_DEFAULT_REGION=us-east-1
user@LAPTOP-H03GEGCQ:/mnt/c/Users/User$ aws --endpoint-url=http://localhost:4566 s3 ls
user@LAPTOP-H03GEGCQ:/mnt/c/Users/User$ aws --endpoint-url=http://localhost:4566 s3 mb s3://test-bucket
make_bucket: test-bucket
user@LAPTOP-H03GEGCQ:/mnt/c/Users/User$ cat > data.csv <<EOF
student_id,name,course,grade,city
1,John Smith,Data Engineering,90,Kyiv
2,Anna Brown,Python,85,Lviv
3,Michael Green,SQL,78,Odesa
4,Sophia White,AWS,92,Kharkiv
5,David Black,Docker,88,Ternopil
EOF
EOF
user@LAPTOP-H03GEGCQ:/mnt/c/Users/User$ aws --endpoint-url=http://localhost:4566 s3 cp data.csv s3://test-bucket
upload: ./data.csv to s3://test-bucket/data.csv
user@LAPTOP-H03GEGCQ:/mnt/c/Users/User$ aws --endpoint-url=http://localhost:4566 s3 ls s3://test-bucket
2026-05-10 16:37:32      198 data.csv
user@LAPTOP-H03GEGCQ:/mnt/c/Users/User$
```

